

String Interpolation with Template Strings

Document #: P3951R1 [[Latest](#)] [[Status](#)]
Date: 2026-03-15
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 Prior Work in C++](#)
 - [2.2 Prior Art in Other Languages](#)
- [3 Design](#)
 - [3.1 Lexing](#)
 - [3.2 Looking for `:`](#)
 - [3.3 Looking for `{` and `}`](#)
 - [3.4 Handling Macro Expansion](#)
 - [3.5 Lexing Trailing Equals](#)
 - [3.6 Lexing Nested Expressions](#)
 - [3.7 Concatenating Consecutive String Literals](#)
 - [3.8 Data Members](#)
 - [3.9 Interpolation Information](#)
 - [3.10 Examples](#)
 - [3.10.1 Basic Formatting](#)
 - [3.10.2 Highlighting](#)
 - [3.10.3 Structured Logging](#)
 - [3.10.4 printf](#)
 - [3.10.5 SQL Statements](#)
 - [3.10.6 Summary](#)
 - [3.11 The `TemplateString` Concept](#)
 - [3.12 Supporting `gettext`](#)
 - [3.13 Support for User-Defined Literals](#)
 - [3.14 Standard Library Design](#)
 - [3.15 Implementation Experience](#)
 - [3.16 More Formal Lexing Specification](#)
- [4 Alternate Approaches](#)
 - [4.1 Object vs Expression-List](#)
 - [4.2 `f-strings`](#)
 - [4.3 Reduced Representation](#)
 - [4.4 Redundant Information](#)
 - [4.5 Static Data Members or Static Member Functions](#)
 - [4.6 Wait for Reflection](#)
- [5 References](#)

§ 1 Revision History

Since [\[P3951R0\]](#):

- More detailed description of the difference between this paper and [\[P3412R3\]](#), including a full section comparing [the object design to the expression-list design](#).
- Support for [user-defined literals](#)
- Added C# and Swift to other string interpolation languages, and a [SQL](#) example.

- Clearer discussion of how to look for [the colon](#) and [the { and }](#).
- Adding discussion of how to support [gettext](#)

§ 2 Introduction

The `std::format` approach to formatting offers many significant benefits over the prior `<iostream>`s approach that need not be revisited here. However, `<iostream>` does still have one significant advantage: ordering. It is easy to see at a glance with a long `std::cout` statement which pieces are to be formatted in which order. With `std::format`, as the amount of replacement fields increases, it becomes increasingly difficult to ensure that they are all correctly ordered.

The solution to this problem is string interpolation: the ability to put the expression to be formatted inside of the format string. This lets us preserve all of the advantages of `std::format`, while also regaining ordering. String interpolation is a wildly popular language feature due to the ease with which it allows users to express complex ideas. It's not surprising that a huge number of modern languages support this to some degree or another. A non-exhaustive list includes: C#, D, Elixir, F#, Groovy, Kotlin, JavaScript, Perl, PHP, Python, Ruby, Rust, Scala, Swift, and VB.

§ 2.1 Prior Work in C++

There have been two prior WG21 papers pursuing string interpolation as a C++ language feature: [\[P1819R0\]\(Interpolated Literals\)](#) and [\[P3412R3\]\(String Interpolation\)](#). The two proposals are quite different, so let's consider an example to work through the details:

```
auto get_result() -> int { return 42; }

auto example() -> void {
    auto interp = f"The result is {get_result()}\n";    // #1
    std::print(interp);                                // #2
    std::print(f"The result is {get_result()}\n");      // #3
}
```

In P1819, the `interp` is an object that is roughly equivalent to:

```
auto interp = [&](auto&& f) -> decltype(auto) {
    return f("The result is ", get_result(), "\n");
};
```

So line #1 does approximately nothing. The call to `get_result()` does not happen yet. Instead, the library would provide new overloads of `std::print` and friends so that in line #2, the library would invoke `interp` with the appropriate function to do the printing. The call to `get_result()` happens at that point. Line #3 does the same things as lines #1 and #2, just together.

In P3412, the behavior is very different. `interp` is already a `std::string`, which is evaluated as:

```
auto interp = std::format("The result is {}\n", get_result());
```

This makes the call in line #2 ill-formed, since `std::print` cannot accept a `std::string`. However, the call in line #3 is valid — by way of a change to overload resolution that recognizes this case as special and instead evaluates the call directly as:

```
std::print("The result is {}\n", get_result());
```

In short, P1819 gives us an object (that doesn't evaluate any of the expressions) while P3412 gives us either a `std::string` or an argument list, depending on context.

Of the two, I think P1819 is significantly better. We get a simple object that can allow for a wide variety of potential functionality. It has two big problems though. The first is that it evaluates lazily and stores its data opaquely — which leads to more surprising behavior, the potential for dangling references, and arbitrarily limited usage. The other is its breakup into pieces doesn't actually play very well with `std::format` — where we would want there to be a format string and we don't have one. The original motivation for the lambda approach

was ease of use — get all the expressions in one convenient format. But the language has evolved since 2019. We have both reflection and packs in structured bindings now, so we don't need the lambda approach anymore.

On the other hand, P3412 is actually not one but two different language features — and it's worth taking some time to evaluate this. This is more explicit in [\[P3412R1\]](#):

Expression	Evaluates As
<code>std::print(x"The result is {get_result()}");</code>	<code>std::print("The result is {}", get_result());</code>
<code>auto s = f"The result is {get_result()}";</code>	<code>auto s = std::format(x"The result is {get_result()}");</code> <code>auto s = std::format("The result is {}", get_result());</code>

The x-literal did string interpolation — it evaluated as an expression-list. That's the workhorse that provides the value of the feature. In contrast, the f-literal was simply syntax sugar for a call to `std::format` with the appropriate x-literal. It's a language feature for simply calling `std::format`. Not precisely `std::format` — since not everybody uses `std::format` so instead this was introduced as a language customization mechanism. But we're really just abbreviating a function call.

In [\[P3412R3\]](#), this becomes significantly more complicated because both of those features (the string interpolation part, and the just-calling-`std::format` part) converge to the same spelling as an f-literal. This is I think inherently suspect because the same expression now means different things in different contexts. Because the spelling is the same, there needs to be a way for the language to differentiate which one the user meant — and that mechanism is overload resolution coupled very strongly to the current implementation strategy of formatting. The call

```
std::print(f"The result is {get_result()}");
```

Works by relying on the first parameter to `std::print` having a consteval constructor. But what if someday we get `constexpr` function parameters and it turns out to be better to implement `basic_format_string<char, Args...>` as taking a `constexpr` `string_view` instead of it being a `consteval` constructor? What if we someday get a different/better macro system such that `std::print("x={}", x)` evaluates not as a call to a function template but rather directly as the expression `std::vprint(validate_fmt_string<int>("x={}", x), std::make_format_args(x))`?

It's not infeasible that some future language change gives us a better way to solve this problem. But with the P3412R3 design, we wouldn't be able to adopt those changes to the formatting functions because they would break string interpolation (unless we come up with a new, more complicated interpolation design, which would now have to recognize multiple implementation strategies).

All this complexity buys us is the ability to create a `std::string` in a single character. However, I don't think that's even a good goal for C++ — we shouldn't hide an operation as costly as string formatting in a single character — and spelling `std::format` is not itself a huge burden. Now, without that aspect of the design, the P3412 approach of having string interpolation emit an expression-list is a lot simpler — I will do a comparison of the two approaches [later in this paper](#).

Instead, this paper proposes an idea much closer to the P1819 model.

§ 2.2 Prior Art in Other Languages

Python 3.6 introduced literal string interpolation (`f"..."`) in [\[PEP-498\]](#), which was later extended in Python 3.14 by template strings (`t"..."`) in [\[PEP-750\]](#). The former directly produces a `string`, while the latter gives a template string — an object with enough information in it to be formatted later.

Rust's `format_args!` is similar to Python's template string — it gives you a completely opaque object (unlike Python's which is completely specified). Both languages gives you a facility to take an interpolated string and produce an object for future work (similar to P1819).

JavaScript also has [template literals](#), which support tagging. A tagged template literal is quite similar to what [\[P3412R3\]](#) proposes:

Code	Evaluates as
<code>myTag`That \${person} is a \${age}.`</code>	<code>myTag(["That ", " is a ", "."], person, age)</code>

This would be similar to P3412’s having `myTag(f"That {person} is a {age}")` evaluate the transformed call `myTag("That {} is a {}", person, age)`. Here, the literal is *not* an object.

C#’s [interpolated strings](#) can produce a string directly. But if they are bound to a `FormattableString`, you can get the string parts and objects separately for future work. Similar to Python template strings and Rust’s facility, except type erased. C# also has a more complicated facility on top of string interpolation called an [InterpolatedStringHandler](#), which allows for more efficient and even conditional (lazy) formatting.

Swift’s string interpolation, similar to C#, can also either directly produce a string or go through a separate, builder path: the protocol `ExpressibleByStringInterpolation`. That [allows](#) the expression:

Code	Evaluates as
<pre>let value: MyType = "Hello, \(name)!"</pre>	<pre>var builder = MyType.StringInterpolation(literalCapacity: 8, interpolationCount: 1) builder.appendLiteral("Hello, ") builder.appendInterpolation(name) builder.appendLiteral("!") let value = MyType(stringInterpolation: builder)</pre>

This paper’s design is along the lines of Python’s template strings, Rust’s `format_args!`, and C#’s `FormattableString` idea. Just presented in a package that is more, well, C++.

§ 3 Design

This paper proposes that we introduce string interpolation for C++ following the same idea as Python’s template strings. We’ll have to come up with a better name than “template string” for this, but for now I’m going to stick with it. A template string literal will eagerly evaluate all of the expressions and produce a new object with all of the relevant pieces, such that it will be suitable for both formatting APIs (like `std::format` and `std::print`) and other APIs that have nothing to do with formatting.

Our example from earlier:

```
auto get_result() -> int { return 42; }

auto example() -> void {
    auto interp = t"The result is {get_result()}\n";
}
```

will evaluate as:

```
auto example() -> void {
    struct Template {
        static constexpr auto fmt() -> char const* { return "The result is {}\n"; }
        static constexpr auto num_interpolations() -> size_t { return 1; }
        static constexpr auto string(size_t i) -> char const* {
            constexpr char const* data[] = {"The result is ", "\n"};
            return data[i];
        }
        static constexpr auto interpolation(size_t i) -> std::interpolation {
            constexpr std::interpolation data[] = {"get_result()", "{}", 0, 1};
            return data[i];
        }

        int _0;
    };

    auto interp = Template{get_result()};
}
```

Let’s go through all of these pieces in order. A template string will generate an instance of a not-necessarily-unique type (unlike a lambda expression, which always has a unique type) by parsing the replacement fields of the string literal. The parsing logic here is very basic and

does not need to understand very much about either the format specifier mini-language or C++ expressions more broadly. The interpolation type will have five public pieces of information in it:

- `fmt()` is a static member function that returns the format string,
- `num_interpolations()` is a static member function that returns the number of interpolations (possibly 0),
- `string(i)` is a static member function that returns the *i*th string part
- `interpolation(i)` is a static member function that returns the *i*th [interpolation information](#), and then lastly
- one non-static [data member](#) for each (possibly-nested) expression. The name of the member is unspecified, but the order and types are.

With this structure, we can easily add additional overloads to the format library to handle template strings:

```
template <TemplateString S>
auto print(S&& s) -> void {
    auto& [...exprs] = s;
    std::print(s.fmt(), exprs...);
}
```

Note that there is no difference in handling between lines #1–2 and line #3 in the example (like the P1819 design and unlike the P3412 one). A template string is just an object, that contains within it all the relevant information. So whether we construct the object separately doesn't matter.

The rest of the paper will go through details on first on how the parsing works and then into other examples to help motivate the structure.

Keep in mind that since a template string object is *just an object*, where most of the information are static data members, this ends up being a very embedded-friendly design too.

§ 3.1 Lexing

A template string is conceptually an alternating sequence of string literals and interpolations. The string literal parts are just normal string literals — we look ahead until we find a non-escaped `{` to start the next replacement field ("`{{`" in a format string is used to print the single character `'{'`). A replacement-field comes in two forms:

```
replacement-field:
{ expr }
{ expr : format-spec }
```

For example, a template string like `t"The price of {id:x} is {price}."` needs to be lexed into these five pieces:

String Literal	"The price of "
Interpolation	"{:x}" with expression id
String Literal	" is "
Interpolation	"{}" with expression price
String Literal	","

The first and last piece are always (possibly-empty) string literals — for *N* interpolations there will be *N*+1 strings. Even for the template string `t"{expr}"` which entirely consists of an expression, there will be two empty string pieces.

However, C++ expressions can be arbitrary complicated. Notably, they can also include `:` or `}`, both of which are significant in formatting and indicate the end of the expression. So how do we know when we're done with the *expr* part here?

One approach would be to simply limit the kinds of expressions that can appear in template strings. Rust, for instance, *only* supports identifiers. That obviously makes parsing quite easy, but it also is very limiting. On the other extreme, supporting *all* expressions can easily lead to indecipherable code. I think on balance, supporting only identifiers is far too restrictive. But once you start adding what other kinds of expressions to allow (surely, at least class member access), it quickly becomes too difficult to keep track of what is allowed (indexing? function calls? splices?) and ironically makes both the implementation more difficult (to enforce what is and isn't allowed) and harder to understand for the user (to know which expressions are and aren't allowed).

I think it's best to simply allow anything in the expression (as Python does) and trust the user to refactor their expressions to be as legible as they desire. This allows us to take a very simple approach: we simply lex *expr* as a balanced token sequence (just counting `{}`s, `()`s, and `[]`s), so that colons and braces inside of any of the bracket kinds are treated as part of an expression. But the first `:` or `}` encountered when we're at a brace depth of zero means we're done with *expr*.

Here are some examples of template string formatting calls and how they would be evaluated. The first column will show a template string that consists entirely of an expression. The second column will show how the format string for that expression will be lexed and the third column will show the expression.

template string	lexed format string	lexed expression
<code>t"{x}"</code>	<code>"{}"</code>	<code>x</code>
<code>t"[]{ return 42; }()"</code>	<code>"{}"</code>	<code>[] { return 42; }()</code>
<code>t"{co_await f(x) + g(y) / z}"</code>	<code>"{}"</code>	<code>co_await f(x) + g(y) / z</code>
<code>t"{a::b}"</code>	<code>"{:b}"</code>	<code>a</code>
<code>t"{{a::b}}"</code>	<code>"{}"</code>	<code>a::b</code>
<code>t"{cond ? a : b}"</code>	<code>"{:b}"</code>	<code>cond ? a</code> ❌
<code>t"{{cond ? a : b}}"</code>	<code>"{}"</code>	<code>(cond ? a : b)</code>

Two important things to note here. First, it is possible to lex an invalid expression due to finding a `:` first, as in the penultimate line. The lexing won't know about this though.

§ 3.2 Looking for `:`

The first problem we run into, as evidenced in the above table, is deciding what it means to find the separator between the *expression* and the *format-spec*. How do we find the `:`? At this point, we will be lexing an *expression* — and there are multiple ways in which a colon can appear here:

- `:`
- `::`
- `[:`
- `:]`
- `<:` (the digraph for `[`)
- `:>` (the digraph for `]`)
- `%:` (the digraph for `#`)
- `\u003a` (the UCN for `:`)

One consideration is what format specifiers exist today. Notably, `"{:x}"` is a valid format specifier (e.g. used to format a range of integers in hex) and `"{:>5}"` is a valid format specifier (e.g. used to right-align a field with width 5). Indeed, any sequence of characters *can* be a valid format specifier. I think it is important that any existing format specifier be usable with string interpolation, otherwise users would find it surprising when something they try to do randomly doesn't work.

To this end, I think the right design is to look for the *character* `:`, not the *token* `:`. That is, `"{a::b}"` should lex as the expression `a`, not `a::b`. If the latter is desired, it has to be parenthesized. This is a difference in the logic proposed in [\[P3412R3\]](#), which looks specifically for the *token* (not character) `:`. This is important because it allows for the most functionality:

```
namespace v { int x = 2; }

auto example() -> void {
    std::vector<int> v = {10, 20, 30};
    std::println(t"{v::x}");    // [a, 14, 1e]
    std::println(t"{{v::x}}");  // 2
}
```

In order to format expressions that use scoping or the conditional operator, you'll just have to write parentheses. That seems easy enough to both understand and use. Otherwise, anything goes.

Now, this approach incurs the burden that you just have to parenthesize any expression with top-level scoping. But the benefit is that all format specifiers just work — we have a formatting design that is flexible, so it would be nice not restrict that. The logic laid out in P3412 effectively forbids any format specifier that starts with a `:` since that `::` would always be interpreted as a scope operator.

An alternative rule would: look for the character `:`, unless it's the token `::`, *except* when it is immediately following a `)`. That would lead to this behavior:

template string	lexed format string	lexed expression
<code>t"{a::b}"</code>	<code>"{}"</code>	<code>a::b</code>
<code>t"{{(a)::b}"</code>	<code>"{:b}"</code>	<code>(a)</code>

This raises the question of how to handle whitespace like `t"{{(a) ::b}"`. It's a more complex rule, and it depends on how frequently we expect top-level `::` and how good the error recovery is. It might be worthwhile, since I would expect top-level scoping to be significantly more common than having a format specifier that starts with a colon. Note that expressions like `decltype(a)::b` exist, which would have to be top-level parenthesized too, but that's a rare construction, so requiring it to be parenthesized is at best a minor inconvenience.

Lastly, there's the question of UCNs. I'll deal with that in its own section.

§ 3.3 Looking for { and }

As with the question of `:`, there are multiple different ways to spell `{` and `}`, because of course there is:

Character	Digraph	UCN
<code>{</code>	<code><%</code>	<code>\u007b</code>
<code>}</code>	<code>%></code>	<code>\u007d</code>

Which of these spellings can start a replacement-field and which of these spellings can end one? Let's start with other languages. Given a variable `v` with value `42`, what happens in...

Language	Expression	Result
Python	<code>f"\u007bv\u007d"</code>	<code>"{v}"</code>
Rust	<code>format!("{}", v);</code>	<code>"42"</code>
Ruby	<code>"#\u007bv\u007d"</code>	<code>"#{v}"</code>

In Rust, interpolation only accepts *identifiers*. But in Python and Ruby, *any* expression can be interpolated, the same as I'm proposing here. This makes the question of looking for `}` more complicated, since what do you do when you see the `\`? In an expression context, UCNs *can* appear — but not to name characters in the basic character set. Like `}`.

To me, the `{` and `}` that start and end a replacement-field is more akin to the braces around a compound-statement (in which a UCN is not allowed) than braces within a string. Which I suppose is an argument for supporting the digraph spellings too. But why? What is the point of supporting this? The goal is to be legible.

To that end, this paper proposes that only literally the characters `{` and `}` start and end a replacement-field. Not the digraphs `<%` and `%>` and not the UCNs `\u007b` and `\u007d`. It is certainly implementable to support a different option. But there isn't a benefit to doing it, and other languages don't either.

§ 3.4 Handling Macro Expansion

While lexing expressions, macro expansion occurs too (although at a later phase, [see below](#)). This is both what users expect and is important to support, otherwise we're not actually meeting the claim of supporting all expressions. There are even standard utilities that are defined as macros, like `errno`. The one thing to note is that looking for the terminating `:` or `}` of an expression will *not* consider such a character from macros. Otherwise, the macros wouldn't actually be usable properly and also the format string itself would become illegible.

Extending the previous example:

```
namespace v { int x = 2; }

auto example() -> void {
    std::vector<int> v = {10, 20, 30};
    std::println(t"{v::x}");    // [a, 14, 1e]
```

```
std::println(t"{{(v::x)}}"); // 2

#define SCOPED v::x
std::println(t"{{SCOPED}}"); // 2, not [a, 14, 1e]
}
```

§ 3.5 Lexing Trailing Equals

One nice debugging feature that Python’s f-strings (and template strings) have is the equals suffix:

```
>>> f"{x=}, {y=}, {z=}"
"x=5, y=7, z='hello world'"
```

Concretely, an expression that ends with an `=` (surrounded by any amount of whitespace) has that suffix appended to the previous string literal piece. Occasionally, there are requests to support `std::print(x, y, z)` to just concatenate those three elements — but that’s not as useful as it initially seems since you quickly forget what variables you’re printing in what order. The ability to support this on the other hand is *very* useful for debugging:

```
std::println(t"{x=}, {y=}, {z=?}");
```

It is also quite easy to implement, since it’s just a matter of checking if the last lexed token of the expression was an `=`. And, if so, dropping that from the expression (since no valid C++ expression ends with `=`) and instead adding the stringified expression to the previous string part.

Concretely: `t"Hello {name=}"` behaves exactly equivalently to `t"Hello name={name}"`.

Note that this isn’t *quite* what Python does — as you might notice from the Python example. In Python, it behaves like `t"Hello name={name!r}"`, which is basically calling `repr(name)` instead of `str(name)`. It would be really nice if we actually had a real answer for debug formatting — but neither `std::format` nor `fmt::format` really have one. There is a `?` specifier which is used to help ensure that range formatting properly works ([P2286R8]), but it’s not valid across all types, and there’s no special handling for it. So we can’t really make `t"{name=}"` evaluate as `t"name={name:?}"`, since that would only work for a small set of types. Instead, this paper proposes not to add any format specifier here.

§ 3.6 Lexing Nested Expressions

Consider the template string:

```
t"{name:>{width}}"
```

There are two expressions here: `name` and `width`. Or rather, we know `name` is an expression, but how do we know that `width` is? In the format model, the format specifiers can be *anything*. There really are no rules — as long as the type’s formatter can handle it. Having nested braces in a format specifier commonly refers to another argument, but it need not actually mean that.

In my CppCon 2022 talk [The Surprising Complexity of Formatting Ranges](#), I work through an example of how one might add underlying specifiers to `std::pair` and `std::tuple`, [where](#):

```
int main() {
    auto elems = std::tuple(10, 20, 30);
    fmt::print("{}\n", elems);           // (10, 20, 30)
    fmt::print("{:x}{#x}{-^4}\n", elems); // (a, 0x14, -30-)
    fmt::print("{:x}{x}{x}\n", elems);   // (a, 20, 1e)
}
```

There, `{x}` doesn’t refer to the expression `x`, it was just chosen as notational convenience. So how can we get this to work?

```
fmt::print(t"{elems:{x}{x}}\n");
```


The short answer is: we cannot. We need to make sense of the template string literal separate from type information, and even if we had type information, it's not like we have a way for the formatter API to signal when it's expecting an expression. We just have to make a choice up front for what to do here. I think we have three options:

- 1. We could explicitly opt *in* to a nested expression. That is, maybe something like `t"{elems:{x}{}${x}}"` signals that the first `{x}` is just a string but the second `{x}` is actually the expression `x`. So in the above example, we simply wouldn't use the `$`.
- 2. We could explicitly opt *out* of being a nested expression with added escaping. Which in this case would be `t"{elems:{{x}}{{}} {{x}}}"`.
- 3. We could say that in a format specifier, encountering a `{` *always* begins an expression that ends at `:` or `}` (same as top-level) and that there is neither opt-in nor opt-out. Meaning that the approach to format specifiers that I demonstrated in that talk wouldn't work, and would instead have to be `t"{elems:{:x}{}{:x}}"`.

I think the third option here is the best. It is at most a minor burden on users as I doubt this approach is in widespread use, and allows for a design that is as simple as possible.

Getting back to our original example:

```
t"{name:>{width}}"
```

This would lex as:

String Literal	""
Interpolation	"{:>{}}" with two expressions: name and width
String Literal	""

And [this works](#) because we recognize `{:x}` as not being a nested expression:

```
fmt::print(t"as template: {elems:{:x}{}{:x}}\n"); // as template: (a, 20, 1e)
```

If we simply stored the expressions and the format string, that would be straightforward: we just have two [data members](#). But we can do better than that. But before we get into the [interpolation information](#), I'll talk about the data members.

§ 3.7 Concatenating Consecutive String Literals

Consecutive string literals are concatenated during preprocessing. The same should hold true for template string literals — which can be concatenated with each other and also with regular string literals, in any order. In the table below, imagine we are initializing a variable `s` to the token sequence shown in the first column and examining the resulting `s.fmt()` and the expressions being lexed:

Tokens	s.fmt()	expressions
"Hello, " "World"	n/a	n/a
"Hello, " t"{name}"	"Hello, {}"	1: name
t"{greeting}, " t"{name}"	"{}", {}"	2: greeting, name
t"{greeting}, " "World"	"{}", World"	1: greeting
t"{greeting}, " "{}"	"{}", {}"	1: greeting ❌

Note the last line. We're concatenating a template string literal and a regular string literal — that simply concatenates the contents of the 2nd string literal onto the last string piece of the 1st — there is no implicit escaping of the braces, so the resulting format string would be incomplete — it has 2 replacement fields but only one expression.

We could consider implicitly escaping the braces for regular string literals that are concatenated to template string literals. I don't know if that's a good idea though.

§ 3.8 Data Members

For any expression, `E` (including nested expressions — so there may be more expressions than interpolations), a non-static data member will be generated (and then initialized from `E`) having type `decltype((E))`. This ensures that we get the right type, but also that we're not

copying anything unnecessarily. For instance:

```
auto verb() -> std::string;

auto example(std::string const& name, std::string relation) -> void {
    auto tmp1 = t"Hello, my name is {name:?}. You killed my {relation}. Prepare to {verb()}.";
}
```

The object `tmp1` will have three members:

- the first, corresponding to `name`, has type `std::string const&`
- the second, corresponding to `relation`, has type `std::string&`
- the third, corresponding to `verb()`, has type `std::string`

These are all public, non-static data members. If we want to copy all of the members (e.g. because we want to serialize them), we can easily do so. It's just that there is no need for the template string itself to do so directly. Note that the call to `verb()` happens immediately and the result is stored in `tmp1`. We are not lazily holding onto the expression `verb()`.

This *does* open up the opportunity for dangling if you write something like this:

```
auto oops(int value) { return t"{value}"; }
```

That template string object will have an `int&` member, refer to the parameter that will be destroyed when we return from the function. I don't think this is a huge use-case of template strings, but it's something to keep in mind. The `decltype((E))` logic is essential for ensuring no overhead for template strings in the expected use-cases. We could consider something like a *leading* = to capture by value instead of by reference, but users can already write `t{auto(value)}` there too.

§ 3.9 Interpolation Information

A template string will have a `static constexpr` member function which returns objects of type `std::interpolation`, which is a simple aggregate with four members:

```
struct interpolation {
    char const* expression;
    char const* fmt;
    size_t index;
    size_t count;
};
```

`expression` is a string literal that is the stringified version of the expression (before macro expansion). `fmt` is the full format specifier with the expression removed, that you would need in order to format this specific expression. Then, we need both `index` and `count` in order to support nested replacement expressions, as we just went through. That gives us both the first non-static data member and the amount of non-static data members this interpolation is associated with.

The template string literal `t"{name=:>{width}}"` would generate the object

```
struct Template {
    static constexpr auto fmt() -> char const* { return "name={:>{}}"; }
    static constexpr auto num_interpolations() -> size_t { return 1; }
    static constexpr auto string(size_t n) -> char const* {
        constexpr char const* data[] = {"name=", ""};
        return data[n];
    }
    static constexpr auto interpolation(size_t n) -> std::interpolation {
        constexpr std::interpolation data[] = {{
            .expression = "name", // note that "name" is preserved
            .fmt = "{:>{}}",      // but "width" is not
            .index = 0,
            .count = 2
        }};
    };
};
```

```

        return data[n];
    }

    std::string const& _0;
    int const& _1;
};

```

Why do we go through the trouble of proving `string(n)` and `interpolation(n)`? Let's look at some examples...

§ 3.10 Examples

There are many things you can do with a template string, so let's just run through them.

§ 3.10.1 Basic Formatting

Consider again our example from earlier:

```

auto tpl = t"Hello, my name is {name}. You killed my {relation}. Prepare to {verb():*^{width}}.";

```

I already showed how we could print this normally, which didn't use either of those two arrays:

```

template <TemplateString S>
auto print(S&& s) -> void {
    auto& [...exprs] = s;
    std::print(s.fmt(), exprs...);
}

```

All of the regular formatting facilities (`std::format`, `std::format_to`, `spdlog::info`, etc.) can be implemented like this. But they could also be implemented a little bit differently — by doing the type-checking internally and calling, e.g., `vprint_unicode` directly. An abbreviated libc++ implementation of `println` might look like this:

```

template <TemplateString S>
auto println(S&& s) -> void {
    auto& [...exprs] = s;
    [[maybe_unused]] constexpr auto check = std::format_string<decltype(exprs)...>(s.fmt());
    std::__print::__vprint_unicode(stdout, s.fmt(), std::make_format_args(exprs...), true);
}

```

§ 3.10.2 Highlighting

But we could do a lot more with this object other than just print it. We could write a function to automatically highlight the interpolations green and bold, which is supported by the `fmt` library but not the standard. That's straightforward, since we're not tied into any particular library, and we have all the information we need:

```

template <TemplateString S>
auto highlight_print(S&& s) -> void {
    constexpr size_t N = s.num_interpolations();

    auto& [...exprs] = s;

    template for (constexpr int I : std::views::indices(N)) {
        fmt::print(s.string(I));

        constexpr auto interp = s.interpolation(I);
        constexpr auto [...J] = std::make_index_sequence<interp.count>();
        fmt::print(fmt::emphasis::bold | fg(fmt::color::green),
            interp.fmt,
            exprs...[interp.index + J]...);
    }
}

```

```

    fmt::print(s.string(N));
}

```

§ 3.10.3 Structured Logging

A completely different example would be to turn it into the JSON object `{"name": "Inigo Montoya", "relation": "father", "verb()": "die"}` for use as structured logging:

```

template <TemplateString S>
auto into_json(S&& s) -> boost::json::object {
    auto& [...exprs] = s;

    boost::json::object o;
    template for (constexpr int I : std::views::indices(N)) {
        constexpr auto interp = s.interpolation(I);
        o[interp.expression] = exprs...[interp.index];
    }
    return o;
}

```

§ 3.10.4 printf

Or we could use entirely different formatting mechanisms. We could make this work with `printf` too! This isn't a complete implementation, but should give a sense of what's possible:

```

template <TemplateString S>
auto with_printf(S&& s) -> void {
    constexpr char const* fmt_string = [&]() consteval {
        std::string fmt = s.string(0);
        auto nsdms = nonstatic_data_members_of(remove_cvref(^S), std::meta::access_context::current());

        for (size_t i = 0; i < s.num_interpolations(); ++i) {
            // in theory, this logic would be more interesting
            auto interp = s.interpolation(i);
            auto type = remove_cvref(type_of(nsdms[interp.index]));
            if (type == ^int) {
                fmt += "%d";
            } else {
                if (interp.fmt == std::string_view("{:?}",)) {
                    fmt += "\"%s\"";
                } else {
                    fmt += "%s";
                }
            }

            fmt += s.string(i + 1);
        }

        return std::define_static_string(fmt);
    }();

    auto adjust = [<class T>(T const& arg){
        if constexpr (std::same_as<T, std::string>) {
            return arg.c_str();
        } else {
            return arg;
        }
    };

    auto& [...exprs] = s;
    constexpr auto [...Is] = std::make_index_sequence<s.num_interpolations()>();
    std::printf(fmt_string, adjust(exprs...[s.interpolation(Is).index])...);
}

```

§ 3.10.5 SQL Statements

One of the most famous SQL injection examples is, of course, [Little Bobby Tables](#). We can use template strings to make it easy to build up a statement properly. This example uses [SQLiteCpp](#), but the same idea can be used for any other SQL library really. All you need to know about the library is that it works as follows (assuming we have a `std::string` name):

Current Library Usage	Using Template Strings
<pre>SQLite::Statement query(db, "SELECT * FROM test WHERE name = ?"); query.bind(1, name);</pre>	<pre>auto query = SQLite::makeStatement(db, t"SELECT * FROM test WHERE name = {name}");</pre>

We can provide a nice API for it like this:

```
template <TemplateString S>
auto makeStatement(Database& db, S&& s) -> Statement {
    constexpr char const* sanitized_fmt = [&]() consteval {
        std::string fmt = s.string(0);
        for (size_t i = 0; i < num_interpolations(); ++i) {
            // every interpolation is just turned into ?
            fmt.push_back('?');

            // ... while the string parts are preserved
            fmt.append(s.string(i + 1));
        }
        return std::define_static_string(fmt);
    }();

    auto query = Statement(db, sanitized_fmt);

    auto& [...exprs] = s;
    template for (constexpr int I : std::views::indices(s.num_interpolations())) {
        // could make different choices here: do we want to just use the value
        // or do we want to format it with its provided specifiers? presumably the
        // SQL library author would know what the right thing to do here is
        query.bind(I + 1, exprs...[s.interpolation(I).index]);
    }

    return query;
}
```

And now you get the same nice string formatting syntax for SQL queries as you do for strings.

§ 3.10.6 Summary

The important thing is to expose all the relevant information to users to let them do whatever they want with it. Note that `highlighted_print` uses the `fmt`, `index`, and `count` fields of the interpolation, since it is formatting all of them, but not the expression field. Meanwhile, `into_json` uses only expression and index — it doesn't need any of the format specifier logic, since it isn't actually doing formatting. The `printf` example uses `string` to build up its own format specifier. The SQL likewise builds up its own formatter, and binds arguments with a different API.

And of course, *all* of these operations can be performed on the same object (which is particularly useful in the case of wanting to do both regular and structured logging):

```
auto verb() -> std::string { return "die"; }

auto main() -> int {
    std::string name = "Inigo Montoya";
    int width = 5;

    auto msg = t"Hello, my name is {name:?.}. You killed my {relation}. Prepare to
    {verb():*^{width}}.\n";
    std::print(msg);
    highlighted_print(msg);
    std::println("{} ", into_json(msg));
}
```

```
with_printf(msg);
}
```

will print (note that `highlighted_print` uses the format specifiers, so the name is quoted):

```
Hello, my name is "Inigo Montoya". You killed my father. Prepare to *die*.
Hello, my name is "Inigo Montoya". You killed my father. Prepare to *die*.
{"name":"Inigo Montoya","relation":"father","verb()":"die"}
Hello, my name is "Inigo Montoya". You killed my father. Prepare to die.
```

Having both `interp.index` and `interp.count` is a little clunky, especially since `interp.count` will almost always be `1`. But I think it's better to put the clunkiness there and maintain the trivial formatting implementations (where you can just unpack the template string object).

You can see this example on [compiler explorer](#). Note that the implementations there are slightly different, since Clang doesn't yet implement `constexpr` structured bindings and the implementations of pack indexing and expansion statements had a few bugs so I came up with workarounds.

§ 3.11 The TemplateString Concept

In these examples, I've been using this `TemplateString` concept to identify a template string object. The question is, what does that concept look like? This one I'm not sure about yet. It can't be a built-in, since users might need to create one of these objects. Consider a logger:

```
log::info(t"Got a trade for in {symbol}: {side} {qty} @ {price}");
```

The expressions here are all cheap to copy, but formatting is expensive — so I might want to serialize all of the data into a background thread to do my formatting there. I don't want to just copy the template string object, since it might have references. With reflection, I can create a new type that has only value members, and then keep the same `fmt`, `strings`, and `interpolations`:

```
template <TemplateString S, class F>
auto map(S s, F f) {
    struct Base;
    consteval {
        vector<meta::info> specs;
        for (meta::info m : nonstatic_data_members_of(^S)) {
            specs.push_back(data_member_spec({
                .type=invoke_result(^F, {type_of(m)}),
                .name=identifier_of(m),
            }));
        }
        define_aggregate(^Base, specs);
    }
    struct R : Base {
        static constexpr auto fmt = S::fmt;
        static constexpr auto string = S::string;
        static constexpr auto interpolation = S::interpolation;
        static constexpr auto num_interpolations = S::num_interpolations;
    };

    auto& [...pieces] = s;
    return R{{f(FWD(pieces))...}};
}
```

Which allows the implementation of all of the logging functions to map their provided template string object to decay or otherwise transform every member into something that won't dangle.

I'd want to make sure this `R` here is also considered a template string for all of these purposes. There are a few options here, not really sure which would be best:

- an attribute

- an annotation
- structural conformance: simply check for the presence of `fmt`, `string`, `num_interpolations`, and `interpolation`?

§ 3.12 Supporting `gettext`

One question that comes up is the question of translation. How do we support `gettext`? This is a difficult one for me to answer given my complete lack of any familiarity with `gettext`, but my understanding is there are two aspects to support: the tooling to pull out strings from the source code and the runtime translation layer.

On the tooling side, without interpolation, `_("Hello {}")` is easily toolable since the macro wraps a string literal and this can be pulled out. However, with interpolation, `_(t"Hello {name}")` now wraps an *object* (or an [expression-list](#)), and either way that is more difficult to deal with. The preprocessing step will have to produce something useful here — or the tooling itself will have to parse the interpolated string. Now, the parsing rules aren't very complicated, so that's not a huge problem — but there will have to be changes on that side no matter what.

On the runtime translation side, it raises the question of how exactly to design this facility. This paper throughout assumes that a template string object, `t"..."`, is produced by the implementation, and thus has its format string as a constant (proposed specifically as a [static consteval](#) function returning a `char const*`). Of course, if we're doing runtime translation, we're not going to end up with a constant... anything. If we want to support this syntax:

```
std::println(_(t"You have {count} new messages."));
```

Then the result is that macro has to take in a template string object and produce a new object that `std::println` understands and can directly format. Note that this differs from the [gettext guidance](#) which tells you to use `std::vformat` directly instead of `std::format`. I'd like to aim higher.

Now, the implementation I showed for `std::print` earlier was:

```
template <TemplateString S>
auto println(S&& s) -> void {
    auto& [...exprs] = s;
    std::println(s.fmt(), exprs...);
}
```

This... doesn't actually *require* `fmt()` to be a constant. It either has to be constant or be the result of a call to `std::runtime_format`. One option ([example](#)) is to have the `gettext` macro do something like this:

```
template <std::template_string S>
auto translate(S ts, Language lang) {
    struct Translated {
        std::string_view translated_fmt;
        S s;

        auto fmt() const { return std::runtime_format(translated_fmt); }
        auto exprs() const -> S const& { return s; }
    };

    return Translated{
        .translated_fmt = lookup_translation(ts.fmt(), lang),
        .s = std::move(ts),
    };
}
```

And thus have two layers of concept:

Dynamic Template String	Static Template String
-------------------------	------------------------

— <code>s.fmt()</code> returns either <code>char const*</code> or <code>std::runtime_format</code> — <code>s.exprs()</code> returns some destructurable object	— <code>S::fmt()</code> must be static and return a constant <code>char const*</code> — <code>S::string(n)</code> must be static and return a constant <code>char const*</code> — <code>S::num_interpolations()</code> must be static and return a constant <code>size_t</code> — <code>S::interpolation(n)</code> must be static and return a constant interpolation — <code>s.exprs()</code> returns some destructurable object
---	---

Note that the static template string concept would subsume the dynamic one. The formatting use-cases (like `std::format`, `std::print`, and `spdlog::info`) only require a dynamic template string. Some of the more complex ones (like SQL and `printf`) require a static template string.

There is something nice about this model. We make the objects more opaque. Instead of talking about the non-static data members of template string objects, we instead just say there's an `exprs()` member function that gives them to you. And this approach does make the run-time support for `gettext` (and possibly similar kinds of transformations) fairly straightforward, as you can see here and on compiler explorer.

But also it adds inherent complexity. Now we have two kinds of template string objects that users have to reason about. But as far as I can tell, the only way to support runtime translation like this is to either introduce two kinds of template string objects or simply weaken the only one. Which reduces functionality.

I cannot answer this question without a better understanding of `gettext`, how important it is to support for string interpolation, and whether this approach I've come up with even supports it.

For this problem, [P3412R3] has an easier path to supporting `gettext`, since in that paper an f-literal is an expression-list, and so it should be possible to preprocess your way to wrapping just the format string part. Given a function `translate` which returns a call to `std::runtime_format`, this would work:

Example Preprocessing Implementation	Preprocesses Into
<pre>#define _2(fmt, ...) \ translate(fmt) __VA_OPT__(,) __VA_ARGS__ #define _(...) _2(__VA_ARGS__) #define FSTRING "You have {} new messages.", count std::println(_(FSTRING));</pre>	<pre>std::println(translate("You have {} new messages.") , count);</pre>

§ 3.13 Support for User-Defined Literals

Consider the expression:

```
auto msg = t"The result is {get_result()}\n"s;
```

Does this make sense to support? It has obvious meaning — evaluate as the call `operator"" s(t"The result is {get_result()}\n")`. And that literal operator can be defined to call `std::format`. The rule in 12.6 [over.literal] would have to be extended to support this signature:

```
template <class T>
auto operator"" ud-suffix(T ) -> R;
```

That actually gives people the terse string construction that they want, *without* an additional language feature that needs to be customized.

The rules of which user-defined literal operator is even looked for is based on the kind of the literal. So `t"x={x}"s` will never even attempt to look for the existing `operator"" s` and, similarly, adding a new one to the standard library will never affect code that currently does `"hello"s`, since the literal operator template would not even be a candidate.

This extension is very narrow, and is effectively specific to template strings.

§ 3.14 Standard Library Design

For the standard library, we need several pieces:

- a `std::template_string` concept, [of some sort](#)
- additional overloads of the formatting function templates (`std::format`, `std::format_to`, `std::format_to_n`, `std::print`, and `std::println`) that take a template string and do the right thing with it.
- a [new UDL](#) for making a string.

The last piece that's potentially worth considering is adding a string constructor. Conceptually, these two declarations are equivalent:

```
auto msg1 = t"The result is {get_result()}\n"s;  
std::string msg2 = t"The result is {get_result()}\n";
```

So is it worth touching `std::string`? I don't think it actually is. In direct code, `std::string` and `std::format` are the same length, so you don't gain anything from having the constructor. The only potential advantage would be calling a function that takes a `std::string` like `f(t"x={x}")` (or similar initialization examples). On the one hand, it doesn't seem completely wrong to add it (since having `s` is conceptually similar) and it's easy enough to implement. On the other, once we add `s`, it hardly seems necessary.

§ 3.15 Implementation Experience

I implemented this in Clang, on top of the p2996 reflection branch. Code can be found in my fork in the `template-strings` branch [here](#) (you can see the diff against p2996 [here](#)) I'm sure there are better ways to do some of what I did. The implementation includes the design laid out in this section, including UDL support, and also the library support — the concept, new overloads of formatting function templates, a new `s` literal operator for string, and a new string constructor (but only for `std::string` specifically).

It can also be used on [compiler explorer](#).

The only difference between this paper and the implementation is that instead of introducing the type `std::interpolation`, I just made `_Interpolation` implicitly defined at global scope for convenience.

This does raise the question of how `std::interpolation` should be defined. Should this facility really require a new header? Maybe it's a sufficiently trivial type (an aggregate with no member functions and just four data members, each of scalar type) that the compiler can just generate it? Maybe we don't care about additional headers because `import std`; anyway?

The same question goes for if we want to implement the [concept](#) by way of annotation. That annotation would be an empty type, could the compiler just create it?

§ 3.16 More Formal Lexing Specification

In Clang, preprocessing happens during lexing — and so the way I implemented it was that a template string lexes as a new kind of token — really a meta-token that itself contains a bunch of other tokens. That's fine as far as Clang goes (or maybe not, there might be a more preferred approach), but the standard defines the phases of translation (5.2 [\[lex.phases\]](#)) in a strict order. Specifically:

- Phase 3: Source file decomposed into preprocessing tokens
- Phase 4: Preprocessing directives and macro expansion occurs
- Phase 5: String-literal encoding and concatenation
- Phase 6: Tokens
- Phase 7: The rest of the owl

The wording needs to fit this specification. Which we can do by defining a new set of preprocessing tokens. For example, the template string

```
t"My name is {name():>{width}}.\n"
```

can lex into the tokens (indentation for clarity):

```
template-string-begin
  string-literal "My name is "

  template-string-interpolation-begin
    string-literal "{:>{}"
    '
    string-literal "name()"

    template-string-expression-begin
      identifier "name"
      (
      )
    template-string-expression-end

    template-string-expression-begin
      identifier "width"
    template-string-expression-end
  template-string-interpolation-end

  string-literal ".\n"
template-string-end
```

That is, we have a bunch of meta-tokens that don't lead to any output and are just there to guide parsing. A template string literal lexes into this alternating sequence of *string-literals* and interpolations. An interpolation is bounded by *template-string-interpolation-begin* and *template-string-interpolation-end*, starts with two *string-literals* for the format string and expression (with a separator token just to avoid string concatenation from kicking in — I used `,` above but the actual separator doesn't matter), and then continues with at least one expression. An expression is bounded by *template-string-expression-begin* and *template-string-expression-end*.

To handle [trailing =](#), we could go one of two ways. Consider just `t"Got {x = }."`. That could be either:

Pre-emptively append	Delayed concat
<pre>template-string-begin string-literal "Got x = " template-string-interpolation-begin string-literal "{}" ' string-literal "x" template-string-expression-begin identifier "name" template-string-expression-end template-string-interpolation-end string-literal "." template-string-end</pre>	<pre>template-string-begin string-literal "Got " // the initial string string-literal "x = " // expression with trailing = template-string-interpolation-begin string-literal "{}" ' string-literal "x" template-string-expression-begin identifier "name" template-string-expression-end template-string-interpolation-end string-literal "." template-string-end</pre>

It works either way. With delayed concatenation, we rely on phase 5 to concatenate the string literals before parsing anyway.

We could conceivably also just introduce a single token and use braces for everything else. Though probably better to use something other than braces to make it easier to diagnose poorly formatted expressions? Whatever people prefer:

Several tokens for clarity	Just one token
<pre>template-string-begin string-literal "My name is " template-string-interpolation-begin string-literal "{:>{}" ' string-literal "name()" template-string-expression-begin identifier "name"</pre>	<pre>template-string { string-literal "My name is " { string-literal "{:>{}" ' string-literal "name()" { identifier "name"</pre>

Several tokens for clarity	Just one token
<code>(</code>	<code>(</code>
<code>)</code>	<code>)</code>
<code>template-string-expression-end</code>	<code>}</code>
<code>template-string-expression-begin</code>	<code>{</code>
<code>identifier "width"</code>	<code>identifier "width"</code>
<code>template-string-expression-end</code>	<code>}</code>
<code>template-string-interpolation-end</code>	<code>}</code>
<code>string-literal ".\n"</code>	<code>string-literal ".\n"</code>
<code>template-string-end</code>	<code>}</code>

§ 4 Alternate Approaches

This proposal is definitely not the only way to do string interpolation in C++. I've [already discussed](#) two previous proposals in this space and why I think what I'm proposing is a better design. But it's worth talking about other approaches as well.

§ 4.1 Object vs Expression-List

As I mentioned earlier, P3412 is really two language features: a string interpolation feature whose intermediate representation is an expression-list, and a feature which just calls `std::format` on that expression-list. In contrast, this paper is a string interpolation feature whose intermediate representation is an object. How do those two intermediate representations compare?

When it comes to formatting specifically — when the sink algorithm is `std::print` or `std::format_to` or `spdlog::info` or anything like that — the object approach is pure overhead. Being able to write `spdlog::info(f"x={x}")` and have that evaluate exactly as `spdlog::info("x={}", x)` means that no library change is necessary whatsoever in order for libraries to “adopt” string interpolation. You can't beat zero work. With this paper, there would have to be library opt-in. Those opt-ins are going to be very simple — mostly two liners as you can see from the print implementation [earlier](#) — but they still have to exist.

So is it worth the added complexity of the object model to justify the added cost of the interpolation opt-in? We have to talk about the added functionality.

Because the object approach preserves all the information in the original format string, you can do things like structured logging (as in the JSON example from earlier). The expression-list approach simply doesn't have the “names” of the expressions anymore, so they're not available for further use.

But almost anything else you might want to do with the interpolated string that isn't precisely formatting is much easier when you have an object. It's actually still surprising to me that some of these things are even possible, but let's walk through some examples. For logging, I might want to also include the file/line number as part of the message. I have this information available at compile-time along with the format string, so it'd be nice to concatenate those together. That's possible:

Expression-list (P3412)	Object (this paper)
<pre> template <class... Args> struct fmt_string_sloc { char const* fmt; template <class S> requires std::convertible_to<S, std::string_view> constexpr fmt_string_sloc(S s, std::source_location sloc = std::source_location::current()) { // have to do type-checking here [[maybe_unused]] auto _ = std::format_string<Args...>(s); this->fmt = std::define_static_string(std::format("[{}:{}] {}", sloc.file_name(), sloc.line(), std::string_view(s))); } }; </pre>	<pre> template <std::template_string S> auto print_sloc2(S&& s) -> void { constexpr auto fmt = []{ using T = std::remove_cvref_t<S>; auto sloc = source_location_of(^T); return std::define_static_string(std::format("[{}:{}] {}", sloc.file_name(), sloc.line(), T::fmt())); }(); // type-checking happens out here // as usual // since fmt is a constant </pre>

Expression-list (P3412)	Object (this paper)
<pre>template <class... Args> auto print_sloc1(fmt_string_sloc<std::type_identity_t<Args>...> fmt, Args&&... args) -> void { std::print(std::runtime_format(fmt.fmt), args...); }</pre>	<pre>auto& [...exprs] = s; std::print(fmt, exprs...); }</pre>

We cannot provide the source location as a parameter to `print_sloc1`, because it's a variadic function template. And even if we could, it couldn't be a constant. However, we can be clever and provide it as a defaulted parameter to the consteval constructor of the non-deduced `fmt_string_sloc`. With the object, we can just directly get the source location of the string interpolation type.

This difference in complexity goes up really fast once you start doing more interesting things. Consider the `highlight_print` example from earlier. This is actually [implementable](#) in the expression-list model, but not easily:

Expression-list (P3412)	Object (this paper)
<pre>struct Interpolation { char const* fmt; int index; int count; }; struct Information { size_t num_interpolations; char const* const* strings; Interpolation const* interpolations; }; template <Information Info, class... Args> auto highlight_print_impl(fmt::text_style style, Args&&... exprs) -> void { constexpr size_t N = Info.num_interpolations; template for (constexpr int I : std::views::indices(N)) { fmt::print(Info.strings[I]); constexpr auto interp = Info.interpolations[I]; constexpr auto [...J] = std::make_index_sequence<interp.count>(); fmt::print(style, interp.fmt, exprs...[interp.index + J]...); } fmt::print(Info.strings[N]); } template <class T> constexpr auto parse_next_impl(fmt::parse_context<char>& ctx) - > void { fmt::formatter<T> f; auto cur = ctx.begin(); if (*cur == ':') { ++cur; } ctx.advance_to(cur); (void)ctx.next_arg_id(); cur = f.parse(ctx); if (cur != ctx.end()) { ++cur; } ctx.advance_to(cur); } template <class... Ts> consteval auto into_interpolation_info(std::string_view sv) -> std::meta::info { fmt::detail::type types[] = { fmt::detail::mapped_type_constant<Ts, char>::value...}; auto ctx = fmt::detail::compile_parse_context<char>(</pre>	<pre>template <TemplateString S> auto highlight_print(fmt::text_style style, S&& s) -> void { constexpr size_t N = s.num_interpolations(); auto& [...exprs] = s; template for (constexpr int I : std::views::indices(N)) { fmt::print(s.string(I)); constexpr auto interp = s.interpolation(I); constexpr auto [...J] = std::make_index_sequence<interp.count> (); fmt::print(style, interp.fmt, exprs...[interp.index + J]...); } fmt::print(s.string(N)); }</pre>

Expression-list (P3412)	Object (this paper)
<pre> sv, sizeof...(Ts), types, 0); constexpr auto arg_id = nonstatic_data_members_of(^fmt::parse_context<char>, std::meta::access_context::unchecked())[1]; std::vector<char const*> strings; std::vector<Interpolation> interpolations; std::vector<std::meta::info> args = {^Ts...}; while (true) { // next string auto next = std::find(ctx.begin(), ctx.end(), '{'); strings.push_back(std::define_static_string(std::string_view(ctx.begin(), next))); if (next == ctx.end()) { break; } // next interpolation ctx.advance_to(next + 1); int const index = ctx[:arg_id:]; auto parse_fn = substitute(^parse_next_impl, {args[index]}); extract<auto*>(fmt::parse_context<char>&)->void> (parse_fn)(ctx); int const count = ctx[:arg_id:] - index; interpolations.push_back({ .fmt = std::define_static_string(std::string_view(next, ctx.begin())), .index=index, .count=count }); } auto info = Information{ .num_interpolations = interpolations.size(), .strings = std::define_static_array(strings).data(), .interpolations = std::define_static_array(interpolations).data(), }; return std::meta::reflect_constant(info); } template <class... Ts> struct highlight_format_string { auto (*impl)(fmt::text_style, Ts&&...) -> void; template <class S> requires std::convertible_to<S, std::string_view> constexpr highlight_format_string(S str) { [[maybe_unused]] auto check = fmt::format_string<Ts...> (str); impl = extract<auto*>(fmt::text_style, Ts&&...)->void> (substitute(^highlight_print_impl, { into_interpolation_info<std::remove_cvref_t<Ts>...>(std::string_view(str)), ^Ts... })); } }; template <class... Ts> auto highlight_print(fmt::text_style style, </pre>	

Expression-list (P3412)	Object (this paper)
<pre>std::type_identity_t<highlight_format_string<Ts...>> fmt, Ts&&... args) -> void { fmt.impl(style, (Ts&&)args...); }</pre>	

The implementation on the left reuses `{fmt}` implementation details, to avoid having to re-implement parsing on my own — that’s just to save some effort, it’s not strictly necessary. The fact that it’s implementable at all is kind of incredible (thanks to Reflection), but the difference in complexity here is pretty vast. But this is because we have to basically re-implement interpolation in user-space and then come up with a clever way have that still work during constant evaluation time. Once we actually do all that work, we can produce the same representation, so the actual interesting part (`highlight_print_impl` on the left) looks the same as it does on the right. But you have to do all that work first.

So the comparison boils down to this:

- for vanilla formatting examples, the expression-list approach requires no library changes at all, so is a clear win over the object approach
- as the examples drift further away from straightforward formatting, the object approach gets steadily more advantageous — since you already have all the information as constants in a single location, you don’t need to come up with a clever solution to make this work
- any structured logging use-cases that might require the names of the expressions is impossible in the expression-list approach.

It depends on how interested we are in all of those other use-cases. I can’t promise that none of them will ever be useful, so it seems like a good forward-looking trade-off to me.

§ 4.2 f-strings

Given that this paper proposes [UDL support](#), there doesn’t seem to be any reason to add an additional language feature to directly produce a `std::string`:

```
auto a = std::format("My name is {} and my age next year is {}", name, age+1); // status quo
auto b = std::format(t"My name is {name} and my age next year is {age+1}"); // proposed
auto c = f"My name is {name} and my age next year is {age+1}"; // not
proposed
auto d = t"My name is {name} and my age next year is {age+1}"s; // proposed
auto e = std::string(t"My name is {name} and my age next year is {age+1}"); // not
proposed
std::string f = t"My name is {name} and my age next year is {age+1}"; // not
proposed
```

Sure, using `d` requires a `using namespace std::literals;` or a more specific one somewhere, but if that level of terseness is desired, it’s worth it. Certainly doesn’t seem worth it for me to have a dedicated language feature for this.

§ 4.3 Reduced Representation

What should this template string literal evaluate to?

```
t"New connection on {ip:#x}:{port}"
```

This paper proposes the one on the left, but we could just do the one on the right:

Proposed	Simpler
<pre>struct S { static constexpr auto fmt() -> char const*; static constexpr auto num_interpolations() -> size_t; static constexpr auto string(size_t n) -> char const*; static constexpr auto interpolation(size_t n) -> interpolation;</pre>	<pre>struct S { static constexpr auto fmt() -> char const*;</pre>

Proposed	Simpler
<pre>u32 _0; u16 _1; };</pre>	<pre>u32 _0; u16 _1; };</pre>

But the main motivation (and likely the most common use-case) is some version of formatting, for which all we need is `S::fmt()`. We wouldn't need `S::interpolation(n)` or `S::string(n)`. Should we still generate the extra functions?

I would argue that we should. The implementation has to do all the work to get those pieces anyway (with the exception of the index and count members for each interpolation, which really isn't much work), so it's not like we're saving much in the way of computation by stripping the interface. The simpler interface is only simple in that it reduces the available functionality. Doesn't seem like a good idea.

§ 4.4 Redundant Information

Continuing with the previous example, what if instead of removing `S::string` and `S::interpolation`, we instead removed `S::fmt`?

Proposed	Minimal
<pre>struct S { static constexpr auto fmt() -> char const*; static constexpr auto num_interpolations() -> size_t; static constexpr auto string(size_t n) -> char const*; static constexpr auto interpolation(size_t n) -> interpolation; u32 _0; u16 _1; };</pre>	<pre>struct S { static constexpr auto num_interpolations() -> size_t; static constexpr auto string(size_t n) -> char const*; static constexpr auto interpolation(size_t n) -> interpolation; u32 _0; u16 _1; };</pre>

Note that `S::fmt()` is exactly the result of concatenating `S::string(0)`, `S::interpolation(0).fmt`, `S::string(1)`, `S::interpolation(1).fmt`, and `S::string(2)`. This is true by construction for all template strings, and is precisely how it is implemented as well. Given that `S::string(n)` and `S::interpolation(n)` will both exist (as being more fundamental), do we need to also provide `S::fmt()` — which can simply be derived from both arrays?

One advantage of removing `S::fmt()` is to avoid having that string spill into the binary even if unused, if the implementation simply fails to detect its lack of use. However, I think we should. While formatting will not be the *only* usage of these objects, it is both the main motivating and primary one, so it will both be more convenient for users and more efficient if the compiler simply does that little bit of extra work to produce the full format string as well.

And regardless, the opposite problem would still exist anyway — if only `S::fmt()` were used, there is the potential that the string literals in `S::string(n)` and `S::interpolation(n)` spill into the binary unnecessary as well.

§ 4.5 Static Data Members or Static Member Functions

The proposal right now has 4 static member functions. But a simple alternative would be to instead provide three, with `strings()` and `interpolations()` returning `std::spans` instead of having a `string(n)` and `interpolation(n)` and `num_interpolations()`:

```
struct S {
    static constexpr auto fmt() -> char const*;
    static constexpr auto strings() -> std::span<char const* const>;
    static constexpr auto interpolations() -> std::span<std::interpolation const>;
};
```

The advantage of approach is that it allows directly looping over the interpolations via:

```
template for (constexpr auto interp : s.interpolations()) {
    // ...
}
```

Which instead some of the [examples](#) work around via:

```
template for (constexpr auto I : std::views::indices(s.num_interpolations())) {
    constexpr auto interp = s.interpolation(I);
}
```

The disadvantage is that it brings in the `std::span` dependency. But if we're declaring `std::interpolation` anyway, that's probably not that big a deal.

A very different shape might instead be to have static *data members* instead of static functions, where:

```
struct S {
    static constexpr char const* fmt = /* ... */;
    static constexpr char const* strings[] = /* ... */;
    static constexpr std::interpolation interpolations[] = /* ... */;
};
```

This would avoid the span dependency and avoid a bunch of parentheses that you would have to write, as compared to the other function version. However, it has two problems. First, a template string can have no interpolations, and we still don't have zero-sized arrays in C++. We ran into this problem with specifying `std::meta::reflect_constant_array`, and it's very annoying that it doesn't just work (even as gcc and clang happily support them with expected semantics with no warnings). Neither the version where `interpolations()` returns a `std::span` nor the version where we have `interpolation(n)` which returns the *n*th interpolation have this problem.

The second problem is that these types may have to be local types in some contexts, and local types cannot have static data members in C++. I do not know why local types have this restriction, given that local types are allowed to have static member functions and those member functions are allowed to have static local variables. But the restriction does currently exist.

§ 4.6 Wait for Reflection

The evergreen question with proposals like this is to wonder if we should wait for reflection. There even is a proposal, [\[P3294R2\]](#) ([Code Injection with Token Sequences](#)), that has walks through how a future macro could solve this problem. Concretely, we would need:

- macros that can inject token sequences,
- the ability to invoke such a macro with a string literal (or `string_view`), and
- the ability to turn a `string_view` into a token sequence

Given those pieces, the implementation of something like Rust's `format_args!` is basically the same as how you would implement it in a compiler. And the benefit of being able to do this in a library is pretty clear: it is much easier to experiment with different functionality. Plus, this would just be a very small taste of what code injections could do.

So should we wait? It seems incredibly unlikely that we will land something as expansive as token sequence injection in C++29 (if ever?), and a dedicated language feature for template string objects is pretty small and self-contained.

§ 5 References

[P1819R0] Vittorio Romeo. 2019-07-20. Interpolated Literals.

<https://wg21.link/p1819r0>

[P2286R8] Barry Revzin. 2022-05-16. Formatting Ranges.

<https://wg21.link/p2286r8>

[P3294R2] Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. 2024-10-15. Code Injection with Token Sequences.

<https://wg21.link/p3294r2>

[P3412R1] Bengt Gustafsson, Victor Zverovich. 2025-01-12. String interpolation.

<https://wg21.link/p3412r1>

[P3412R3] Bengt Gustafsson, Victor Zverovich. 2025-12-14. String Interpolation.

<https://wg21.link/p3412r3>

[P3951R0] Barry Revzin. 2026-01-10. String Interpolation Objects.

<https://wg21.link/p3951r0>

[PEP-498] Eric V. Smith. 2015-08-01. Literal String Interpolation.

<https://peps.python.org/pep-0498/>

[PEP-750] Jim Baker, Guido van Rossum, Paul Everitt, Koudai Aono, Lysandros Nikolaou, and Dave Peck. 2024-07-08. Template Strings.

<https://peps.python.org/pep-0750/>